# SONA COLLEGE OF TECHNOLOGY

**(An Autonomous Institution)**

## Department of Computer Science & Engineering

### Real Time Problem Solving

**U23CS307 – Object Oriented Programming Using Java Laboratory**

| | |
|---|---|
| **Academic Year** | 2026-2027 (Odd) |
| **Year / Sem / Section** | II / III / C |
| **Project Title** | Vehicle Rental Management System |
| **Name and Reg No. of the Student** | Samritha S - 2503617810422152<br>Sakthivel V - 2503617810421150<br>Sam Gladson A - 2503617810431151<br>Sandhya Yadav – 2503617810422153<br>Rakshitha S - 2503617810422138 |
| **Faculty Name** | Dr. P. Thiyagarajan, CSE<br>Ms. S. Lavanya, CSE |
| **Signature** | |

## TABLE OF CONTENTS


| S.No | Title | Page No |
|---|---|---|
| 1 | Introduction | 1 |
| 2 | Description | 2 |
| 3 | Key Features | 3 |
| 4 | Program | 4 - 25 |
| 5 | Output | 26 - 37 |
| 6 | Conclusion | 38 |

# 1. Introduction

The Vehicle Rental Management System is a Java-based console application designed to help a rental business manage its fleet of cars, motorcycles and trucks, its registered customers, and all of its rental transactions from one place. Instead of recording bookings in notebooks or loose spreadsheets, the operator uses a simple menu-driven terminal interface to add vehicles, register customers, book rentals, take vehicles back and view reports.

The system automatically checks whether a vehicle is available before it is rented, calculates the rental amount (including optional insurance), adds a late fee when a vehicle is returned after the due date, and prints a receipt for every transaction. All information is stored in CSV files so that the data remains available the next time the program is run.

The main objective of this project is to apply core Object Oriented Programming concepts of Java, such as classes and objects, inheritance, abstract classes, interfaces, polymorphism, encapsulation, exception handling, collections and file handling, to solve a real-time problem in a clean and well-organised manner.

# 2. Description

The Vehicle Rental Management System is developed in Java using only the standard library. It runs entirely in the terminal: the user chooses options from numbered menus and types the required details, and the program responds with formatted tables, messages and receipts. No graphical interface or external library is needed.

The fleet is modelled with an abstract class Vehicle that implements the Rentable interface. The classes Car, Motorcycle and Truck extend it and add their own details, such as seating capacity and fuel type for cars, engine capacity for motorcycles, and load capacity for trucks. Customers and rental transactions are represented by the Customer and Rental classes, and enumerations are used for vehicle type and for vehicle and rental status.

The program follows a layered design. The model package holds the data classes, the storage package reads and writes the CSV files, the service package contains the business rules (availability checks, pricing, late fees, revenue calculation), and the cli package handles menus and validated keyboard input. Custom exceptions report problems such as an unavailable vehicle or an unknown record without stopping the program.

A rental is priced as the daily rate multiplied by the number of days, plus an optional per-day insurance charge that depends on the vehicle type. If a vehicle is returned after its due date, a late fee of 1.5 times the daily rate is charged for every extra day, and any damage or cleaning charge entered by the operator is added to the final bill. Cancelling an active booking makes the vehicle available again.

The Reports menu gives a fleet summary with utilisation, and a revenue report grouped by vehicle category that can also be exported to a text file. Sample vehicles and customers are created the first time the program is run, so that every feature can be tried immediately.

# 3. The Key Features

1. Vehicle Fleet Management: Add cars, motorcycles and trucks, view the whole fleet or only available vehicles, update daily rates, and remove vehicles that are not currently rented.

2. Customer Management: Register customers with validated name, mobile number, e-mail and driving licence number, search customers, and view the complete rental history of any customer.

3. Rental Booking: Select an available vehicle and a customer, choose the start date, number of days, insurance option and payment method, and confirm the booking after seeing the estimated cost.

4. Return and Late Fee Calculation: Complete a rental by entering the return date, distance driven and any damage or cleaning charges; a late fee of 1.5 times the daily rate is added for every day after the due date.

5. Booking Cancellation: Cancel an active booking so that the vehicle immediately becomes available for other customers.

6. Maintenance Tracking: Send a vehicle to maintenance or bring it back into service; vehicles under maintenance or already rented cannot be booked.

7. Search and View Records: Search vehicles by id, brand, model or number plate, search customers by id, name, phone or licence, and list active or all rentals in neat tables.

8. Receipt Generation: A detailed receipt showing the rental period, base charge, insurance, late fee, extra charges, total amount and payment method is printed for every booking and return.

9. Reports: Fleet summary with utilisation percentage and a revenue report by vehicle category, with an option to export the reports to a text file.

10. Data Storage, Validation and Exception Handling: All records are saved in CSV files after every change; keyboard input is validated and custom exceptions handle errors so the program never crashes on wrong input.

# 4. Java program used in the system

## 1. Main Application Entry Point (Main.java)

```
package com.rtps.vehiclerental;

import com.rtps.vehiclerental.cli.ConsoleUI;
import com.rtps.vehiclerental.service.CustomerService;
import com.rtps.vehiclerental.service.RentalService;
import com.rtps.vehiclerental.service.VehicleService;
import com.rtps.vehiclerental.storage.DataStore;

import java.util.NoSuchElementException;

/** Entry point of the RTPS Vehicle Rental Management System (terminal version). */
public class Main {
    public static void main(String[] args) {
        DataStore store = new DataStore("data");
        VehicleService vehicleService = new VehicleService(store);
        CustomerService customerService = new CustomerService(store);
        RentalService rentalService = new RentalService(store, vehicleService, customerService);

        ConsoleUI ui = new ConsoleUI(vehicleService, customerService, rentalService);
        try {
            ui.run();
        } catch (NoSuchElementException e) {
            // input stream closed (Ctrl+D / Ctrl+Z) - leave quietly
            System.out.println("\nInput closed. Data saved. Goodbye!");
        }
    }
}
```

## 2. Rentable Interface (model/Rentable.java)

```
package com.rtps.vehiclerental.model;

/** Contract for anything that can be handed out on rent and taken back. */
public interface Rentable {
    boolean isAvailable();

    void markRented();

    void markReturned();
}
```

## 3. VehicleType Enumeration (model/VehicleType.java)

```
package com.rtps.vehiclerental.model;

/** Vehicle categories supported by the system, with the id prefix and insurance rate of each. */
public enum VehicleType {
    CAR("Car", "CAR", 300.0),
    MOTORCYCLE("Motorcycle", "BIKE", 100.0),
    TRUCK("Truck", "TRK", 500.0);

    private final String label;
    private final String idPrefix;
    private final double insuranceRatePerDay;

    VehicleType(String label, String idPrefix, double insuranceRatePerDay) {
        this.label = label;
        this.idPrefix = idPrefix;
        this.insuranceRatePerDay = insuranceRatePerDay;
    }

    public String getLabel() { return label; }
    public String getIdPrefix() { return idPrefix; }
```

```
    public double getInsuranceRatePerDay() { return insuranceRatePerDay; }

    @Override
    public String toString() { return label; }
}
```

## 4. VehicleStatus Enumeration (model/VehicleStatus.java)

```
package com.rtps.vehiclerental.model;

/** Operational status of a vehicle in the fleet. */
public enum VehicleStatus {
    AVAILABLE("Available"),
    RENTED("Rented"),
    MAINTENANCE("Maintenance");

    private final String label;

    VehicleStatus(String label) { this.label = label; }

    public String getLabel() { return label; }

    @Override
    public String toString() { return label; }
}
```

## 5. RentalStatus Enumeration (model/RentalStatus.java)

```
package com.rtps.vehiclerental.model;

/** Life-cycle status of a rental booking. */
public enum RentalStatus {
    ACTIVE("Active"),
    COMPLETED("Completed"),
    CANCELLED("Cancelled");

    private final String label;

    RentalStatus(String label) { this.label = label; }

    public String getLabel() { return label; }

    @Override
    public String toString() { return label; }
}
```

## 6. Abstract Vehicle Class (model/Vehicle.java)

```
package com.rtps.vehiclerental.model;

/**
 * Abstract base class for every vehicle in the fleet.
 * Car, Motorcycle and Truck extend it and supply their own specifications.
 */
public abstract class Vehicle implements Rentable {
    private final String id;
    private final VehicleType type;
    private String brand;
    private String model;
    private int year;
    private String licensePlate;
    private double dailyRate;
    private VehicleStatus status;
    private int odometerKm;

    protected Vehicle(String id, VehicleType type, String brand, String model, int year,
                      String licensePlate, double dailyRate, VehicleStatus status, int odometerKm) {
        this.id = id;
```

```
        this.type = type;
        this.brand = brand;
        this.model = model;
        this.year = year;
        this.licensePlate = licensePlate;
        setDailyRate(dailyRate);
        this.status = status;
        this.odometerKm = odometerKm;
    }

    public String getId() { return id; }
    public VehicleType getType() { return type; }
    public String getBrand() { return brand; }
    public String getModel() { return model; }
    public int getYear() { return year; }
    public String getLicensePlate() { return licensePlate; }
    public double getDailyRate() { return dailyRate; }
    public VehicleStatus getStatus() { return status; }
    public int getOdometerKm() { return odometerKm; }

    public void setDailyRate(double dailyRate) {
        if (dailyRate <= 0) {
            throw new IllegalArgumentException("Daily rate must be greater than zero.");
        }
        this.dailyRate = dailyRate;
    }

    public void setStatus(VehicleStatus status) { this.status = status; }

    public void addDistance(int km) {
        if (km > 0) {
            this.odometerKm += km;
        }
    }

    public String getFullName() { return year + " " + brand + " " + model; }

    /** Each subclass describes its own specifications (polymorphism). */
    public abstract String getSpecifications();

    /** Subclass-specific fields, joined with ';' for CSV storage. */
    protected abstract String extraCsvFields();

    /** Trucks may need a commercial licence; other vehicles do not. */
    public boolean requiresSpecialLicense() { return false; }

    @Override
    public boolean isAvailable() { return status == VehicleStatus.AVAILABLE; }

    @Override
    public void markRented() { this.status = VehicleStatus.RENTED; }

    @Override
    public void markReturned() { this.status = VehicleStatus.AVAILABLE; }

    public String toCsv() {
        return String.join(";", type.name(), id, brand, model, String.valueOf(year), licensePlate,
                String.valueOf(dailyRate), status.name(), String.valueOf(odometerKm),
extraCsvFields());
    }

    @Override
    public String toString() {
        return getFullName() + " [" + licensePlate + "]";
    }
}
```

## 7. Car Class (model/Car.java)

```
package com.rtps.vehiclerental.model;
```

```
/** A passenger car (hatchback, sedan, SUV, ...). */
public class Car extends Vehicle {
    private final int seats;
    private final String fuelType;
    private final String transmission;

    public Car(String id, String brand, String model, int year, String licensePlate, double
dailyRate,
               VehicleStatus status, int odometerKm, int seats, String fuelType, String
transmission) {
        super(id, VehicleType.CAR, brand, model, year, licensePlate, dailyRate, status, odometerKm);
        this.seats = seats;
        this.fuelType = fuelType;
        this.transmission = transmission;
    }

    public int getSeats() { return seats; }
    public String getFuelType() { return fuelType; }
    public String getTransmission() { return transmission; }

    @Override
    public String getSpecifications() {
        return seats + " seats, " + transmission + ", " + fuelType;
    }

    @Override
    protected String extraCsvFields() {
        return seats + ";" + fuelType + ";" + transmission;
    }
}
```

## 8. Motorcycle Class (model/Motorcycle.java)

```
package com.rtps.vehiclerental.model;

/** A two-wheeler. */
public class Motorcycle extends Vehicle {
    private final int engineCc;
    private final boolean helmetIncluded;

    public Motorcycle(String id, String brand, String model, int year, String licensePlate, double
dailyRate,
                      VehicleStatus status, int odometerKm, int engineCc, boolean helmetIncluded) {
        super(id, VehicleType.MOTORCYCLE, brand, model, year, licensePlate, dailyRate, status,
odometerKm);
        this.engineCc = engineCc;
        this.helmetIncluded = helmetIncluded;
    }

    public int getEngineCc() { return engineCc; }
    public boolean isHelmetIncluded() { return helmetIncluded; }

    @Override
    public String getSpecifications() {
        return engineCc + " cc, helmet " + (helmetIncluded ? "included" : "not included");
    }

    @Override
    protected String extraCsvFields() {
        return engineCc + ";" + helmetIncluded;
    }
}
```

## 9. Truck Class (model/Truck.java)

```
package com.rtps.vehiclerental.model;

/** A goods carrier / commercial vehicle. */
public class Truck extends Vehicle {
    private final double capacityTons;
    private final boolean commercialLicenseRequired;
```

```
    public Truck(String id, String brand, String model, int year, String licensePlate, double
dailyRate,
                 VehicleStatus status, int odometerKm, double capacityTons, boolean
commercialLicenseRequired) {
        super(id, VehicleType.TRUCK, brand, model, year, licensePlate, dailyRate, status,
odometerKm);
        this.capacityTons = capacityTons;
        this.commercialLicenseRequired = commercialLicenseRequired;
    }

    public double getCapacityTons() { return capacityTons; }

    @Override
    public boolean requiresSpecialLicense() { return commercialLicenseRequired; }

    @Override
    public String getSpecifications() {
        return capacityTons + " ton capacity" + (commercialLicenseRequired ? ", commercial licence
needed" : "");
    }

    @Override
    protected String extraCsvFields() {
        return capacityTons + ";" + commercialLicenseRequired;
    }
}
```

## 10. Customer Class (model/Customer.java)

```
package com.rtps.vehiclerental.model;

import java.time.LocalDate;

/** A registered customer who can rent vehicles. */
public class Customer {
    private final String id;
    private final String name;
    private final String phone;
    private final String email;
    private final String licenseNumber;
    private final LocalDate registeredOn;

    public Customer(String id, String name, String phone, String email,
                    String licenseNumber, LocalDate registeredOn) {
        this.id = id;
        this.name = name;
        this.phone = phone;
        this.email = email;
        this.licenseNumber = licenseNumber;
        this.registeredOn = registeredOn;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public String getPhone() { return phone; }
    public String getEmail() { return email; }
    public String getLicenseNumber() { return licenseNumber; }
    public LocalDate getRegisteredOn() { return registeredOn; }

    public String toCsv() {
        return String.join(";", id, name, phone, email, licenseNumber, registeredOn.toString());
    }

    @Override
    public String toString() {
        return name + " (" + id + ")";
    }
}
```

## 11. Rental Class (model/Rental.java)

```
package com.rtps.vehiclerental.model;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

/** One rental transaction: who rented what, for how long, and what it costs. */
public class Rental {
    /** Late returns are charged at 1.5 times the daily rate for every extra day. */
    public static final double LATE_FEE_MULTIPLIER = 1.5;

    private final String id;
    private final String customerId;
    private final String customerName;
    private final String vehicleId;
    private final String vehicleName;
    private final VehicleType vehicleType;
    private final LocalDate startDate;
    private final LocalDate expectedReturn;
    private LocalDate actualReturn;
    private final double dailyRate;
    private final int days;
    private final boolean insured;
    private final double insuranceRate;
    private double lateFee;
    private double extraCharges;
    private double total;
    private RentalStatus status;
    private final String paymentMethod;
    private String notes;

    /** Creates a new ACTIVE booking. */
    public Rental(String id, Customer customer, Vehicle vehicle, LocalDate startDate, int days,
                  boolean insured, String paymentMethod) {
        this.id = id;
        this.customerId = customer.getId();
        this.customerName = customer.getName();
        this.vehicleId = vehicle.getId();
        this.vehicleName = vehicle.toString();
        this.vehicleType = vehicle.getType();
        this.startDate = startDate;
        this.days = days;
        this.expectedReturn = startDate.plusDays(days);
        this.dailyRate = vehicle.getDailyRate();
        this.insured = insured;
        this.insuranceRate = insured ? vehicle.getType().getInsuranceRatePerDay() : 0.0;
        this.paymentMethod = paymentMethod;
        this.status = RentalStatus.ACTIVE;
        this.notes = "";
        recalculateTotal();
    }

    /** Restores a rental read from the CSV file. */
    public Rental(String id, String customerId, String customerName, String vehicleId, String
vehicleName,
                  VehicleType vehicleType, LocalDate startDate, LocalDate expectedReturn, LocalDate
actualReturn,
                  double dailyRate, int days, boolean insured, double insuranceRate, double lateFee,
                  double extraCharges, double total, RentalStatus status, String paymentMethod,
String notes) {
        this.id = id;
        this.customerId = customerId;
        this.customerName = customerName;
        this.vehicleId = vehicleId;
        this.vehicleName = vehicleName;
        this.vehicleType = vehicleType;
        this.startDate = startDate;
        this.expectedReturn = expectedReturn;
        this.actualReturn = actualReturn;
        this.dailyRate = dailyRate;
        this.days = days;
        this.insured = insured;
        this.insuranceRate = insuranceRate;
        this.lateFee = lateFee;
        this.extraCharges = extraCharges;
        this.total = total;
        this.status = status;
```

```
        this.paymentMethod = paymentMethod;
        this.notes = notes;
    }

    public double getBaseCost() { return dailyRate * days; }

    public double getInsuranceCost() { return insuranceRate * days; }

    private void recalculateTotal() {
        total = getBaseCost() + getInsuranceCost() + lateFee + extraCharges;
    }

    /** Closes the rental, adding a late fee (if overdue) and any extra charges such as damage or
cleaning. */
    public void complete(LocalDate returnedOn, double extraCharges, String notes) {
        long lateDays = ChronoUnit.DAYS.between(expectedReturn, returnedOn);
        this.lateFee = lateDays > 0 ? lateDays * dailyRate * LATE_FEE_MULTIPLIER : 0.0;
        this.extraCharges = extraCharges;
        this.actualReturn = returnedOn;
        this.notes = notes;
        this.status = RentalStatus.COMPLETED;
        recalculateTotal();
    }

    public void cancel(String reason) {
        this.status = RentalStatus.CANCELLED;
        this.notes = reason;
    }

    public String getId() { return id; }
    public String getCustomerId() { return customerId; }
    public String getCustomerName() { return customerName; }
    public String getVehicleId() { return vehicleId; }
    public String getVehicleName() { return vehicleName; }
    public VehicleType getVehicleType() { return vehicleType; }
    public LocalDate getStartDate() { return startDate; }
    public LocalDate getExpectedReturn() { return expectedReturn; }
    public LocalDate getActualReturn() { return actualReturn; }
    public double getDailyRate() { return dailyRate; }
    public int getDays() { return days; }
    public boolean isInsured() { return insured; }
    public double getInsuranceRate() { return insuranceRate; }
    public double getLateFee() { return lateFee; }
    public double getExtraCharges() { return extraCharges; }
    public double getTotal() { return total; }
    public RentalStatus getStatus() { return status; }
    public String getPaymentMethod() { return paymentMethod; }
    public String getNotes() { return notes; }

    public String toCsv() {
        return String.join(";", id, customerId, customerName, vehicleId, vehicleName,
vehicleType.name(),
                startDate.toString(), expectedReturn.toString(),
                actualReturn == null ? "-" : actualReturn.toString(),
                String.valueOf(dailyRate), String.valueOf(days), String.valueOf(insured),
                String.valueOf(insuranceRate), String.valueOf(lateFee),
String.valueOf(extraCharges),
                String.valueOf(total), status.name(), paymentMethod, notes);
    }
}
```

## 12. RentalException (exception/RentalException.java)

```
package com.rtps.vehiclerental.exception;

/** Thrown when a business rule is broken (vehicle not available, duplicate record, ...). */
public class RentalException extends Exception {
    public RentalException(String message) {
        super(message);
    }
}
```

## 13. RecordNotFoundException (exception/RecordNotFoundException.java)

```
package com.rtps.vehiclerental.exception;

/** Thrown when a vehicle, customer or rental id does not exist. */
public class RecordNotFoundException extends RentalException {
    public RecordNotFoundException(String kind, String id) {
        super(kind + " not found with id: " + id);
    }
}
```

## 14. File Handling and Data Store (storage/DataStore.java)

```
package com.rtps.vehiclerental.storage;

import com.rtps.vehiclerental.model.*;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/**
 * File-handling layer. Keeps the fleet, customers and rentals in memory
 * and saves them to semicolon-separated CSV files after every change.
 * Sample data is created the first time the program runs.
 */
public class DataStore {
    private final Path dir;
    private final Path vehicleFile;
    private final Path customerFile;
    private final Path rentalFile;

    private final List<Vehicle> vehicles = new ArrayList<>();
    private final List<Customer> customers = new ArrayList<>();
    private final List<Rental> rentals = new ArrayList<>();

    public DataStore(String directory) {
        this.dir = Paths.get(directory);
        this.vehicleFile = dir.resolve("vehicles.csv");
        this.customerFile = dir.resolve("customers.csv");
        this.rentalFile = dir.resolve("rentals.csv");
        load();
    }

    public List<Vehicle> getVehicles() { return vehicles; }
    public List<Customer> getCustomers() { return customers; }
    public List<Rental> getRentals() { return rentals; }
    public Path getDirectory() { return dir; }

    /** Removes characters that would break the CSV format. */
    public static String clean(String text) {
        return text == null ? "" : text.replace(";", ",").replace("\n", " ").trim();
    }

    /** Next id such as CAR-107, based on the highest number already used with this prefix. */
    public String nextId(String prefix) {
        int max = 100;
        for (String id : allIds()) {
            if (id.startsWith(prefix + "-")) {
                try {
                    max = Math.max(max, Integer.parseInt(id.substring(prefix.length() + 1)));
                } catch (NumberFormatException ignored) {
                    // not one of our generated ids
                }
            }
        }
        return prefix + "-" + (max + 1);
```

```
    }

    private List<String> allIds() {
        List<String> ids = new ArrayList<>();
        vehicles.forEach(v -> ids.add(v.getId()));
        customers.forEach(c -> ids.add(c.getId()));
        rentals.forEach(r -> ids.add(r.getId()));
        return ids;
    }

    // ------------------------------------------------------------------ loading

    private void load() {
        try {
            Files.createDirectories(dir);
            if (!Files.exists(vehicleFile)) {
                seedSampleData();
                saveAll();
                return;
            }
            for (String line : readLines(vehicleFile)) {
                Vehicle v = parseVehicle(line);
                if (v != null) vehicles.add(v);
            }
            for (String line : readLines(customerFile)) {
                Customer c = parseCustomer(line);
                if (c != null) customers.add(c);
            }
            for (String line : readLines(rentalFile)) {
                Rental r = parseRental(line);
                if (r != null) rentals.add(r);
            }
        } catch (IOException e) {
            System.out.println("Warning: could not read data files (" + e.getMessage() + ").
Starting empty.");
        }
    }

    private List<String> readLines(Path file) throws IOException {
        List<String> lines = new ArrayList<>();
        if (Files.exists(file)) {
            for (String line : Files.readAllLines(file, StandardCharsets.UTF_8)) {
                if (!line.isBlank()) lines.add(line);
            }
        }
        return lines;
    }

    private Vehicle parseVehicle(String line) {
        try {
            String[] p = line.split(";", -1);
            VehicleType type = VehicleType.valueOf(p[0]);
            String id = p[1], brand = p[2], model = p[3], plate = p[5];
            int year = Integer.parseInt(p[4]);
            double rate = Double.parseDouble(p[6]);
            VehicleStatus status = VehicleStatus.valueOf(p[7]);
            int km = Integer.parseInt(p[8]);
            switch (type) {
                case CAR:
                    return new Car(id, brand, model, year, plate, rate, status, km,
                            Integer.parseInt(p[9]), p[10], p[11]);
                case MOTORCYCLE:
                    return new Motorcycle(id, brand, model, year, plate, rate, status, km,
                            Integer.parseInt(p[9]), Boolean.parseBoolean(p[10]));
                default:
                    return new Truck(id, brand, model, year, plate, rate, status, km,
                            Double.parseDouble(p[9]), Boolean.parseBoolean(p[10]));
            }
        } catch (RuntimeException e) {
            System.out.println("Skipping bad vehicle record: " + line);
            return null;
        }
    }

    private Customer parseCustomer(String line) {
        try {
```

```
            String[] p = line.split(";", -1);
            return new Customer(p[0], p[1], p[2], p[3], p[4], LocalDate.parse(p[5]));
        } catch (RuntimeException e) {
            System.out.println("Skipping bad customer record: " + line);
            return null;
        }
    }

    private Rental parseRental(String line) {
        try {
            String[] p = line.split(";", -1);
            LocalDate actual = p[8].equals("-") ? null : LocalDate.parse(p[8]);
            return new Rental(p[0], p[1], p[2], p[3], p[4], VehicleType.valueOf(p[5]),
                    LocalDate.parse(p[6]), LocalDate.parse(p[7]), actual,
                    Double.parseDouble(p[9]), Integer.parseInt(p[10]), Boolean.parseBoolean(p[11]),
                    Double.parseDouble(p[12]), Double.parseDouble(p[13]), Double.parseDouble(p[14]),
                    Double.parseDouble(p[15]), RentalStatus.valueOf(p[16]), p[17], p.length > 18 ?
p[18] : "");
        } catch (RuntimeException e) {
            System.out.println("Skipping bad rental record: " + line);
            return null;
        }
    }

    // ------------------------------------------------------------------ saving

    public void saveAll() {
        try {
            Files.createDirectories(dir);
            List<String> lines = new ArrayList<>();
            vehicles.forEach(v -> lines.add(v.toCsv()));
            Files.write(vehicleFile, lines, StandardCharsets.UTF_8);

            lines.clear();
            customers.forEach(c -> lines.add(c.toCsv()));
            Files.write(customerFile, lines, StandardCharsets.UTF_8);

            lines.clear();
            rentals.forEach(r -> lines.add(r.toCsv()));
            Files.write(rentalFile, lines, StandardCharsets.UTF_8);
        } catch (IOException e) {
            System.out.println("Warning: could not save data (" + e.getMessage() + ")");
        }
    }

    // ------------------------------------------------------------------ sample data

    private void seedSampleData() {
        vehicles.add(new Car("CAR-101", "Maruti Suzuki", "Swift VXi", 2023, "TN30AB1101", 1800,
                VehicleStatus.AVAILABLE, 12400, 5, "Petrol", "Manual"));
        vehicles.add(new Car("CAR-102", "Hyundai", "Creta SX", 2024, "TN30CD2202", 3200,
                VehicleStatus.AVAILABLE, 8900, 5, "Diesel", "Automatic"));
        vehicles.add(new Car("CAR-103", "Toyota", "Innova Crysta", 2022, "TN30EF3303", 4200,
                VehicleStatus.AVAILABLE, 41200, 7, "Diesel", "Manual"));
        vehicles.add(new Car("CAR-104", "Tata", "Nexon EV", 2024, "TN30GH4404", 2900,
                VehicleStatus.MAINTENANCE, 6500, 5, "Electric", "Automatic"));
        vehicles.add(new Motorcycle("BIKE-101", "Royal Enfield", "Classic 350", 2023, "TN30JK5501",
1200,
                VehicleStatus.AVAILABLE, 9800, 349, true));
        vehicles.add(new Motorcycle("BIKE-102", "TVS", "Apache RTR 160", 2024, "TN30LM6602", 700,
                VehicleStatus.AVAILABLE, 4300, 160, true));
        vehicles.add(new Truck("TRK-101", "Mahindra", "Bolero Pickup", 2023, "TN30NP7701", 3500,
                VehicleStatus.AVAILABLE, 28700, 1.5, false));
        vehicles.add(new Truck("TRK-102", "Tata", "Ace Gold", 2022, "TN30QR8802", 2200,
                VehicleStatus.AVAILABLE, 35100, 0.75, false));

        customers.add(new Customer("CUST-101", "Karthik Raja", "9876501234",
"karthik.raja@example.com",
                "TN3020180012345", LocalDate.of(2026, 8, 12)));
        customers.add(new Customer("CUST-102", "Meena Lakshmi", "9843012345", "meena.l@example.com",
                "TN3020190054321", LocalDate.of(2026, 9, 3)));
        customers.add(new Customer("CUST-103", "Arun Prakash", "9790123456", "arun.p@example.com",
                "TN3020170098765", LocalDate.of(2026, 9, 28)));
    }
}
```

## 15. Vehicle Service (service/VehicleService.java)

```
package com.rtps.vehiclerental.service;

import com.rtps.vehiclerental.exception.RecordNotFoundException;
import com.rtps.vehiclerental.exception.RentalException;
import com.rtps.vehiclerental.model.*;
import com.rtps.vehiclerental.storage.DataStore;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/** Business logic for managing the vehicle fleet. */
public class VehicleService {
    private final DataStore store;

    public VehicleService(DataStore store) {
        this.store = store;
    }

    public List<Vehicle> getAll() {
        return new ArrayList<>(store.getVehicles());
    }

    public List<Vehicle> getAvailable() {
        return
store.getVehicles().stream().filter(Vehicle::isAvailable).collect(Collectors.toList());
    }

    public Vehicle findById(String id) throws RecordNotFoundException {
        for (Vehicle v : store.getVehicles()) {
            if (v.getId().equalsIgnoreCase(id.trim())) return v;
        }
        throw new RecordNotFoundException("Vehicle", id);
    }

    /** Case-insensitive search over id, brand, model and number plate. */
    public List<Vehicle> search(String keyword) {
        String k = keyword.trim().toLowerCase();
        return store.getVehicles().stream()
                .filter(v -> v.getId().toLowerCase().contains(k)
                        || v.getBrand().toLowerCase().contains(k)
                        || v.getModel().toLowerCase().contains(k)
                        || v.getLicensePlate().toLowerCase().contains(k))
                .collect(Collectors.toList());
    }

    public String nextId(VehicleType type) {
        return store.nextId(type.getIdPrefix());
    }

    public void add(Vehicle vehicle) throws RentalException {
        for (Vehicle v : store.getVehicles()) {
            if (v.getLicensePlate().equalsIgnoreCase(vehicle.getLicensePlate())) {
                throw new RentalException("A vehicle with number plate " + vehicle.getLicensePlate()
+ " already exists.");
            }
        }
        store.getVehicles().add(vehicle);
        store.saveAll();
    }

    public void updateRate(String id, double newRate) throws RentalException {
        Vehicle v = findById(id);
        v.setDailyRate(newRate);
        store.saveAll();
    }

    /** Moves a vehicle into or out of maintenance. A rented vehicle cannot be changed. */
    public VehicleStatus toggleMaintenance(String id) throws RentalException {
        Vehicle v = findById(id);
        if (v.getStatus() == VehicleStatus.RENTED) {
```

```
            throw new RentalException(v.getId() + " is currently rented out and cannot be sent for
maintenance.");
        }
        v.setStatus(v.getStatus() == VehicleStatus.MAINTENANCE ? VehicleStatus.AVAILABLE :
VehicleStatus.MAINTENANCE);
        store.saveAll();
        return v.getStatus();
    }

    public void remove(String id) throws RentalException {
        Vehicle v = findById(id);
        if (v.getStatus() == VehicleStatus.RENTED) {
            throw new RentalException(v.getId() + " is currently rented out and cannot be
removed.");
        }
        store.getVehicles().remove(v);
        store.saveAll();
    }

    public int count(VehicleStatus status) {
        return (int) store.getVehicles().stream().filter(v -> v.getStatus() == status).count();
    }

    public int count(VehicleType type) {
        return (int) store.getVehicles().stream().filter(v -> v.getType() == type).count();
    }
}
```

## 16. Customer Service (service/CustomerService.java)

```
package com.rtps.vehiclerental.service;

import com.rtps.vehiclerental.exception.RecordNotFoundException;
import com.rtps.vehiclerental.exception.RentalException;
import com.rtps.vehiclerental.model.Customer;
import com.rtps.vehiclerental.storage.DataStore;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/** Business logic for customer registration and lookup. */
public class CustomerService {
    private final DataStore store;

    public CustomerService(DataStore store) {
        this.store = store;
    }

    public List<Customer> getAll() {
        return new ArrayList<>(store.getCustomers());
    }

    public Customer findById(String id) throws RecordNotFoundException {
        for (Customer c : store.getCustomers()) {
            if (c.getId().equalsIgnoreCase(id.trim())) return c;
        }
        throw new RecordNotFoundException("Customer", id);
    }

    public List<Customer> search(String keyword) {
        String k = keyword.trim().toLowerCase();
        return store.getCustomers().stream()
                .filter(c -> c.getId().toLowerCase().contains(k)
                        || c.getName().toLowerCase().contains(k)
                        || c.getPhone().contains(k)
                        || c.getLicenseNumber().toLowerCase().contains(k))
                .collect(Collectors.toList());
    }

    public Customer register(String name, String phone, String email, String licenseNumber) throws
```

```
RentalException {
        for (Customer c : store.getCustomers()) {
            if (c.getLicenseNumber().equalsIgnoreCase(licenseNumber)) {
                throw new RentalException("A customer with licence " + licenseNumber + " is already
registered (" + c.getId() + ").");
            }
        }
        Customer customer = new Customer(store.nextId("CUST"), DataStore.clean(name), phone,
                DataStore.clean(email), DataStore.clean(licenseNumber).toUpperCase(),
LocalDate.now());
        store.getCustomers().add(customer);
        store.saveAll();
        return customer;
    }
}
```

## 17. Rental Service - Booking, Return, Late Fee, Receipt (service/RentalService.java)

```
package com.rtps.vehiclerental.service;

import com.rtps.vehiclerental.exception.RecordNotFoundException;
import com.rtps.vehiclerental.exception.RentalException;
import com.rtps.vehiclerental.model.*;
import com.rtps.vehiclerental.storage.DataStore;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

/** Core workflows: booking a vehicle, taking it back, cancelling, and revenue figures. */
public class RentalService {
    private final DataStore store;
    private final VehicleService vehicles;
    private final CustomerService customers;

    public RentalService(DataStore store, VehicleService vehicles, CustomerService customers) {
        this.store = store;
        this.vehicles = vehicles;
        this.customers = customers;
    }

    public List<Rental> getAll() {
        return new ArrayList<>(store.getRentals());
    }

    public List<Rental> getActive() {
        return store.getRentals().stream()
                .filter(r -> r.getStatus() == RentalStatus.ACTIVE).collect(Collectors.toList());
    }

    public List<Rental> getByCustomer(String customerId) {
        return store.getRentals().stream()
                .filter(r ->
r.getCustomerId().equalsIgnoreCase(customerId)).collect(Collectors.toList());
    }

    public Rental findById(String id) throws RecordNotFoundException {
        for (Rental r : store.getRentals()) {
            if (r.getId().equalsIgnoreCase(id.trim())) return r;
        }
        throw new RecordNotFoundException("Rental", id);
    }

    /** Creates a booking and marks the vehicle as rented. */
    public Rental book(String customerId, String vehicleId, LocalDate start, int days,
                       boolean insured, String payment) throws RentalException {
        Customer customer = customers.findById(customerId);
        Vehicle vehicle = vehicles.findById(vehicleId);
        if (!vehicle.isAvailable()) {
            throw new RentalException(vehicle.getId() + " is not available (status: " +
```

```
vehicle.getStatus() + ").");
        }
        if (days < 1) {
            throw new RentalException("A rental must be for at least one day.");
        }
        Rental rental = new Rental(store.nextId("RENT"), customer, vehicle, start, days, insured,
payment);
        store.getRentals().add(rental);
        vehicle.markRented();
        store.saveAll();
        return rental;
    }

    /** Completes a rental: adds late fee / extra charges, frees the vehicle and updates its
odometer. */
    public Rental returnVehicle(String rentalId, LocalDate returnedOn, double extraCharges,
                                int kmDriven, String notes) throws RentalException {
        Rental rental = findById(rentalId);
        if (rental.getStatus() != RentalStatus.ACTIVE) {
            throw new RentalException(rental.getId() + " is already " + rental.getStatus() + ".");
        }
        if (returnedOn.isBefore(rental.getStartDate())) {
            throw new RentalException("Return date cannot be before the start date (" +
rental.getStartDate() + ").");
        }
        rental.complete(returnedOn, extraCharges, DataStore.clean(notes));
        try {
            Vehicle vehicle = vehicles.findById(rental.getVehicleId());
            vehicle.markReturned();
            vehicle.addDistance(kmDriven);
        } catch (RecordNotFoundException ignored) {
            // vehicle was removed from the fleet; the rental record is still valid
        }
        store.saveAll();
        return rental;
    }

    public void cancel(String rentalId, String reason) throws RentalException {
        Rental rental = findById(rentalId);
        if (rental.getStatus() != RentalStatus.ACTIVE) {
            throw new RentalException(rental.getId() + " is already " + rental.getStatus() + " and
cannot be cancelled.");
        }
        rental.cancel(DataStore.clean(reason));
        try {
            vehicles.findById(rental.getVehicleId()).markReturned();
        } catch (RecordNotFoundException ignored) {
            // vehicle no longer in fleet
        }
        store.saveAll();
    }

    // ---------------------------------------------------------------- reports

    /** Revenue from completed rentals, grouped by vehicle category. */
    public Map<VehicleType, Double> revenueByType() {
        Map<VehicleType, Double> map = new EnumMap<>(VehicleType.class);
        for (VehicleType t : VehicleType.values()) map.put(t, 0.0);
        for (Rental r : store.getRentals()) {
            if (r.getStatus() == RentalStatus.COMPLETED) {
                map.merge(r.getVehicleType(), r.getTotal(), Double::sum);
            }
        }
        return map;
    }

    public double completedRevenue() {
        return revenueByType().values().stream().mapToDouble(Double::doubleValue).sum();
    }

    public double pendingRevenue() {
        return getActive().stream().mapToDouble(Rental::getTotal).sum();
    }

    public double totalLateFees() {
```

```
        return store.getRentals().stream()
                .filter(r -> r.getStatus() ==
RentalStatus.COMPLETED).mapToDouble(Rental::getLateFee).sum();
    }

    public int count(RentalStatus status) {
        return (int) store.getRentals().stream().filter(r -> r.getStatus() == status).count();
    }

    // ------------------------------------------------------------------ receipt

    public String buildReceipt(Rental r) {
        String line = "=".repeat(56);
        String thin = "-".repeat(56);
        StringBuilder sb = new StringBuilder();
        sb.append(line).append('\n');
        sb.append(center("RTPS VEHICLE RENTAL MANAGEMENT SYSTEM")).append('\n');
        sb.append(center("RENTAL RECEIPT")).append('\n');
        sb.append(line).append('\n');
        sb.append(row("Rental ID", r.getId()));
        sb.append(row("Status", r.getStatus().getLabel().toUpperCase()));
        sb.append(row("Customer", r.getCustomerName() + " (" + r.getCustomerId() + ")"));
        sb.append(row("Vehicle", r.getVehicleName()));
        sb.append(thin).append('\n');
        sb.append(row("Start date", r.getStartDate().toString()));
        sb.append(row("Expected return", r.getExpectedReturn().toString()));
        if (r.getActualReturn() != null) sb.append(row("Actual return",
r.getActualReturn().toString()));
        sb.append(row("Duration", r.getDays() + " day(s)"));
        sb.append(thin).append('\n');
        sb.append(row("Daily rate", money(r.getDailyRate())));
        sb.append(row("Base rental", money(r.getBaseCost())));
        sb.append(row("Insurance", r.isInsured()
                ? money(r.getInsuranceCost()) + " (" + money(r.getInsuranceRate()) + "/day)" : "Not
taken"));
        if (r.getLateFee() > 0) sb.append(row("Late fee (1.5x/day)", money(r.getLateFee())));
        if (r.getExtraCharges() > 0) sb.append(row("Damage / cleaning",
money(r.getExtraCharges())));
        sb.append(thin).append('\n');
        sb.append(row("TOTAL AMOUNT", money(r.getTotal())));
        sb.append(row("Payment method", r.getPaymentMethod()));
        if (!r.getNotes().isEmpty()) sb.append(row("Notes", r.getNotes()));
        sb.append(line).append('\n');
        sb.append(center("Thank you for choosing us. Drive safe!")).append('\n');
        sb.append(line);
        return sb.toString();
    }

    public static String money(double amount) {
        return String.format("Rs. %.2f", amount);
    }

    private static String row(String label, String value) {
        return String.format("%-20s: %s%n", label, value);
    }

    private static String center(String text) {
        int pad = Math.max(0, (56 - text.length()) / 2);
        return " ".repeat(pad) + text;
    }
}
```

## 18. Validated Console Input (cli/ConsoleInput.java)

```
package com.rtps.vehiclerental.cli;

import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.util.Scanner;

/** Reads validated values from the terminal. Every method keeps asking until the input is valid. */
public class ConsoleInput {
    private final Scanner scanner = new Scanner(System.in);
```

```
    /** Reads a non-empty line. */
    public String text(String prompt) {
        while (true) {
            System.out.print(prompt);
            String value = scanner.nextLine().trim();
            if (!value.isEmpty()) return value;
            System.out.println("  ! This field cannot be empty.");
        }
    }

    /** Reads a line that may be left blank. */
    public String optionalText(String prompt) {
        System.out.print(prompt);
        return scanner.nextLine().trim();
    }

    public int integer(String prompt, int min, int max) {
        while (true) {
            System.out.print(prompt);
            String value = scanner.nextLine().trim();
            try {
                int number = Integer.parseInt(value);
                if (number >= min && number <= max) return number;
                System.out.println("  ! Please enter a number between " + min + " and " + max +
".");
            } catch (NumberFormatException e) {
                System.out.println("  ! '" + value + "' is not a valid whole number.");
            }
        }
    }

    public double decimal(String prompt, double min, double max) {
        while (true) {
            System.out.print(prompt);
            String value = scanner.nextLine().trim();
            try {
                double number = Double.parseDouble(value);
                if (number >= min && number <= max) return number;
                System.out.println("  ! Please enter a value between " + min + " and " + max + ".");
            } catch (NumberFormatException e) {
                System.out.println("  ! '" + value + "' is not a valid number.");
            }
        }
    }

    /** Reads a yyyy-MM-dd date; pressing Enter accepts the default. */
    public LocalDate date(String prompt, LocalDate defaultValue) {
        while (true) {
            System.out.print(prompt + " [yyyy-MM-dd, Enter = " + defaultValue + "]: ");
            String value = scanner.nextLine().trim();
            if (value.isEmpty()) return defaultValue;
            try {
                return LocalDate.parse(value);
            } catch (DateTimeParseException e) {
                System.out.println("  ! Invalid date. Use the format yyyy-MM-dd, e.g. 2026-10-21.");
            }
        }
    }

    public boolean yesNo(String prompt) {
        while (true) {
            System.out.print(prompt + " (y/n): ");
            String value = scanner.nextLine().trim().toLowerCase();
            if (value.equals("y") || value.equals("yes")) return true;
            if (value.equals("n") || value.equals("no")) return false;
            System.out.println("  ! Please answer y or n.");
        }
    }

    /** Reads a 10-digit Indian mobile number. */
    public String phone(String prompt) {
        while (true) {
            String value = text(prompt);
            if (value.matches("[6-9][0-9]{9}")) return value;
```

```
            System.out.println("  ! Enter a valid 10-digit mobile number.");
        }
    }

    public String email(String prompt) {
        while (true) {
            String value = text(prompt);
            if (value.matches("[\\w.+-]+@[\\w-]+(\\.[\\w-]+)+")) return value;
            System.out.println("  ! Enter a valid e-mail address.");
        }
    }

    /** Reads a vehicle registration plate such as TN30AB1234 (spaces/dashes are removed). */
    public String plate(String prompt) {
        while (true) {
            String value = text(prompt).toUpperCase().replaceAll("[\\s-]", "");
            if (value.matches("[A-Z]{2}[0-9]{1,2}[A-Z]{1,3}[0-9]{4}")) return value;
            System.out.println("  ! Enter a valid number plate, e.g. TN30AB1234.");
        }
    }

    public void pause() {
        System.out.print("\nPress Enter to continue...");
        scanner.nextLine();
    }
}
```

## 19. Terminal Menu Interface (cli/ConsoleUI.java)

```
package com.rtps.vehiclerental.cli;

import com.rtps.vehiclerental.exception.RentalException;
import com.rtps.vehiclerental.model.*;
import com.rtps.vehiclerental.service.CustomerService;
import com.rtps.vehiclerental.service.RentalService;
import com.rtps.vehiclerental.service.VehicleService;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.time.LocalDate;
import java.util.List;
import java.util.Map;

/** Menu-driven terminal interface. All business rules live in the service classes. */
public class ConsoleUI {
    private static final String[] PAYMENTS = {"Cash", "UPI", "Credit Card", "Debit Card"};

    private final VehicleService vehicles;
    private final CustomerService customers;
    private final RentalService rentals;
    private final ConsoleInput in = new ConsoleInput();

    public ConsoleUI(VehicleService vehicles, CustomerService customers, RentalService rentals) {
        this.vehicles = vehicles;
        this.customers = customers;
        this.rentals = rentals;
    }

    public void run() {
        System.out.println("============================================================");
        System.out.println("        RTPS VEHICLE RENTAL MANAGEMENT SYSTEM");
        System.out.println("============================================================");
        boolean running = true;
        while (running) {
            System.out.println("\n--- MAIN MENU ---");
            System.out.println("1. Vehicle Management");
            System.out.println("2. Customer Management");
            System.out.println("3. Rental Operations");
            System.out.println("4. Reports");
            System.out.println("0. Exit");
            switch (in.integer("Enter choice: ", 0, 4)) {
```

```
                case 1: vehicleMenu(); break;
                case 2: customerMenu(); break;
                case 3: rentalMenu(); break;
                case 4: reportMenu(); break;
                default: running = false;
            }
        }
        System.out.println("\nAll data saved. Thank you for using the system. Goodbye!");
    }

    // ============================================================ vehicles

    private void vehicleMenu() {
        while (true) {
            System.out.println("\n--- VEHICLE MANAGEMENT ---");
            System.out.println("1. View all vehicles");
            System.out.println("2. View available vehicles");
            System.out.println("3. Search vehicles");
            System.out.println("4. Add a vehicle");
            System.out.println("5. Update daily rate");
            System.out.println("6. Send to / bring back from maintenance");
            System.out.println("7. Remove a vehicle");
            System.out.println("0. Back");
            int choice = in.integer("Enter choice: ", 0, 7);
            try {
                switch (choice) {
                    case 0: return;
                    case 1: printVehicles(vehicles.getAll()); break;
                    case 2: printVehicles(vehicles.getAvailable()); break;
                    case 3: printVehicles(vehicles.search(in.text("Search (id / brand / model /
plate): "))); break;
                    case 4: addVehicle(); break;
                    case 5:
                        String id = in.text("Vehicle ID: ");
                        double rate = in.decimal("New daily rate (Rs.): ", 1, 100000);
                        vehicles.updateRate(id, rate);
                        System.out.println("Daily rate updated.");
                        break;
                    case 6:
                        VehicleStatus now = vehicles.toggleMaintenance(in.text("Vehicle ID: "));
                        System.out.println("Vehicle status is now: " + now);
                        break;
                    default:
                        String removeId = in.text("Vehicle ID to remove: ");
                        if (in.yesNo("Remove " + vehicles.findById(removeId) + "?")) {
                            vehicles.remove(removeId);
                            System.out.println("Vehicle removed.");
                        }
                }
            } catch (RentalException | IllegalArgumentException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }

    private void printVehicles(List<Vehicle> list) {
        if (list.isEmpty()) {
            System.out.println("No vehicles found.");
            return;
        }
        System.out.println();
        System.out.printf("%-9s %-11s %-30s %-12s %9s %-12s %s%n",
                "ID", "Type", "Vehicle", "Plate", "Rate/Day", "Status", "Specifications");
        System.out.println("-".repeat(122));
        for (Vehicle v : list) {
            System.out.printf("%-9s %-11s %-30s %-12s %9.2f %-12s %s%n", v.getId(), v.getType(),
                    v.getFullName(), v.getLicensePlate(), v.getDailyRate(), v.getStatus(),
v.getSpecifications());
        }
        System.out.println(list.size() + " vehicle(s) listed.");
    }

    private void addVehicle() throws RentalException {
        System.out.println("1. Car   2. Motorcycle   3. Truck");
        VehicleType type = VehicleType.values()[in.integer("Vehicle type: ", 1, 3) - 1];
        String brand = in.text("Brand: ");
```

```
        String model = in.text("Model: ");
        int year = in.integer("Year of manufacture: ", 1990, LocalDate.now().getYear() + 1);
        String plate = in.plate("Number plate: ");
        double rate = in.decimal("Daily rate (Rs.): ", 1, 100000);
        int km = in.integer("Odometer reading (km): ", 0, 2_000_000);
        String id = vehicles.nextId(type);
        Vehicle v;
        if (type == VehicleType.CAR) {
            int seats = in.integer("Seats: ", 2, 12);
            String fuel = in.text("Fuel type (Petrol/Diesel/Electric/CNG): ");
            String gear = in.text("Transmission (Manual/Automatic): ");
            v = new Car(id, brand, model, year, plate, rate, VehicleStatus.AVAILABLE, km, seats,
fuel, gear);
        } else if (type == VehicleType.MOTORCYCLE) {
            int cc = in.integer("Engine capacity (cc): ", 50, 2500);
            v = new Motorcycle(id, brand, model, year, plate, rate, VehicleStatus.AVAILABLE, km, cc,
                    in.yesNo("Helmet included?"));
        } else {
            double tons = in.decimal("Load capacity (tons): ", 0.1, 50);
            v = new Truck(id, brand, model, year, plate, rate, VehicleStatus.AVAILABLE, km, tons,
                    in.yesNo("Commercial licence required?"));
        }
        vehicles.add(v);
        System.out.println("Vehicle added with ID " + id + ".");
    }

    // ============================================================ customers

    private void customerMenu() {
        while (true) {
            System.out.println("\n--- CUSTOMER MANAGEMENT ---");
            System.out.println("1. View all customers");
            System.out.println("2. Search customers");
            System.out.println("3. Register new customer");
            System.out.println("4. View rental history of a customer");
            System.out.println("0. Back");
            int choice = in.integer("Enter choice: ", 0, 4);
            try {
                switch (choice) {
                    case 0: return;
                    case 1: printCustomers(customers.getAll()); break;
                    case 2: printCustomers(customers.search(in.text("Search (id / name / phone /
licence): "))); break;
                    case 3:
                        Customer c = registerCustomer();
                        System.out.println("Customer registered with ID " + c.getId() + ".");
                        break;
                    default:
                        Customer found = customers.findById(in.text("Customer ID: "));
                        System.out.println("Rental history of " + found + ":");
                        printRentals(rentals.getByCustomer(found.getId()));
                }
            } catch (RentalException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }

    private Customer registerCustomer() throws RentalException {
        String name = in.text("Full name: ");
        String phone = in.phone("Mobile number (10 digits): ");
        String email = in.email("E-mail: ");
        String licence = in.text("Driving licence number: ");
        return customers.register(name, phone, email, licence);
    }

    private void printCustomers(List<Customer> list) {
        if (list.isEmpty()) {
            System.out.println("No customers found.");
            return;
        }
        System.out.println();
        System.out.printf("%-9s %-20s %-12s %-28s %-18s %s%n", "ID", "Name", "Phone", "E-mail",
"Licence", "Registered");
        System.out.println("-".repeat(104));
        for (Customer c : list) {
```

```
            System.out.printf("%-9s %-20s %-12s %-28s %-18s %s%n", c.getId(), c.getName(),
c.getPhone(),
                    c.getEmail(), c.getLicenseNumber(), c.getRegisteredOn());
        }
    }

    // ============================================================== rentals

    private void rentalMenu() {
        while (true) {
            System.out.println("\n--- RENTAL OPERATIONS ---");
            System.out.println("1. Book a vehicle");
            System.out.println("2. Return a vehicle");
            System.out.println("3. Cancel a booking");
            System.out.println("4. View active rentals");
            System.out.println("5. View all rentals");
            System.out.println("6. Print receipt");
            System.out.println("0. Back");
            int choice = in.integer("Enter choice: ", 0, 6);
            try {
                switch (choice) {
                    case 0: return;
                    case 1: bookVehicle(); break;
                    case 2: returnVehicle(); break;
                    case 3:
                        String id = in.text("Rental ID to cancel: ");
                        String reason = in.text("Reason: ");
                        rentals.cancel(id, reason);
                        System.out.println("Booking cancelled; vehicle is available again.");
                        break;
                    case 4: printRentals(rentals.getActive()); break;
                    case 5: printRentals(rentals.getAll()); break;
                    default:
System.out.println(rentals.buildReceipt(rentals.findById(in.text("Rental ID: "))));
                }
            } catch (RentalException | IllegalArgumentException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }

    private void bookVehicle() throws RentalException {
        printVehicles(vehicles.getAvailable());
        if (vehicles.getAvailable().isEmpty()) return;
        Vehicle vehicle = vehicles.findById(in.text("\nVehicle ID to rent: "));
        if (!vehicle.isAvailable()) {
            throw new RentalException(vehicle.getId() + " is not available (status: " +
vehicle.getStatus() + ").");
        }

        String custId = in.text("Customer ID (type NEW to register): ");
        Customer customer = custId.equalsIgnoreCase("NEW") ? registerCustomer() :
customers.findById(custId);
        System.out.println("Customer: " + customer);

        if (vehicle.requiresSpecialLicense() && !in.yesNo("This vehicle needs a commercial licence.
Does the customer hold one?")) {
            throw new RentalException("Booking refused: commercial licence is required for " +
vehicle.getId() + ".");
        }

        LocalDate start = in.date("Start date", LocalDate.now());
        int days = in.integer("Number of days (1-90): ", 1, 90);
        boolean insured = in.yesNo("Add insurance at Rs. " + (int)
vehicle.getType().getInsuranceRatePerDay() + "/day?");
        System.out.println("Payment method: 1. Cash  2. UPI  3. Credit Card  4. Debit Card");
        String payment = PAYMENTS[in.integer("Choose: ", 1, 4) - 1];

        double estimate = vehicle.getDailyRate() * days
                + (insured ? vehicle.getType().getInsuranceRatePerDay() * days : 0);
        System.out.println("Estimated total: " + RentalService.money(estimate)
                + " for " + days + " day(s), return on " + start.plusDays(days));
        if (!in.yesNo("Confirm booking?")) {
            System.out.println("Booking abandoned.");
            return;
```

```
        }
        Rental rental = rentals.book(customer.getId(), vehicle.getId(), start, days, insured,
payment);
        System.out.println("\nBooking confirmed!\n");
        System.out.println(rentals.buildReceipt(rental));
    }

    private void returnVehicle() throws RentalException {
        List<Rental> active = rentals.getActive();
        printRentals(active);
        if (active.isEmpty()) return;
        Rental rental = rentals.findById(in.text("\nRental ID being returned: "));
        LocalDate returned = in.date("Return date", LocalDate.now());
        int km = in.integer("Distance driven (km): ", 0, 100000);
        double extra = in.decimal("Damage / cleaning charges (Rs., 0 if none): ", 0, 1_000_000);
        String notes = in.optionalText("Notes (optional): ");
        Rental done = rentals.returnVehicle(rental.getId(), returned, extra, km, notes);
        System.out.println("\nVehicle returned successfully.\n");
        System.out.println(rentals.buildReceipt(done));
    }

    private void printRentals(List<Rental> list) {
        if (list.isEmpty()) {
            System.out.println("No rental records found.");
            return;
        }
        System.out.println();
        System.out.printf("%-10s %-16s %-9s %-11s %-11s %-11s %11s %s%n",
                "Rental", "Customer", "Vehicle", "Start", "Due", "Returned", "Total (Rs.)",
"Status");
        System.out.println("-".repeat(100));
        for (Rental r : list) {
            System.out.printf("%-10s %-16s %-9s %-11s %-11s %-11s %11.2f %s%n", r.getId(),
r.getCustomerName(),
                    r.getVehicleId(), r.getStartDate(), r.getExpectedReturn(),
                    r.getActualReturn() == null ? "-" : r.getActualReturn().toString(),
r.getTotal(), r.getStatus());
        }
    }

    // ================================================================ reports

    private void reportMenu() {
        while (true) {
            System.out.println("\n--- REPORTS ---");
            System.out.println("1. Fleet summary");
            System.out.println("2. Revenue report");
            System.out.println("3. Export both reports to data/report.txt");
            System.out.println("0. Back");
            int choice = in.integer("Enter choice: ", 0, 3);
            switch (choice) {
                case 0: return;
                case 1: System.out.println(fleetReport()); break;
                case 2: System.out.println(revenueReport()); break;
                default:
                    try {
                        Files.write(Paths.get("data", "report.txt"),
                                (fleetReport() + "\n\n" + revenueReport() +
"\n").getBytes(StandardCharsets.UTF_8));
                        System.out.println("Report saved to data/report.txt");
                    } catch (IOException e) {
                        System.out.println("Error: could not write report (" + e.getMessage() +
")");
                    }
            }
        }
    }

    private String fleetReport() {
        int total = vehicles.getAll().size();
        int rented = vehicles.count(VehicleStatus.RENTED);
        double utilisation = total == 0 ? 0 : rented * 100.0 / total;
        StringBuilder sb = new StringBuilder("\n===== FLEET SUMMARY =====\n");
        sb.append(String.format("Total vehicles      : %d%n", total));
        for (VehicleType t : VehicleType.values()) {
            sb.append(String.format("  %-17s : %d%n", t + "s", vehicles.count(t)));
```

```
        }
        sb.append(String.format("Available           : %d%n",
vehicles.count(VehicleStatus.AVAILABLE)));
        sb.append(String.format("Rented              : %d%n", rented));
        sb.append(String.format("In maintenance      : %d%n",
vehicles.count(VehicleStatus.MAINTENANCE)));
        sb.append(String.format("Fleet utilisation   : %.1f%%%n", utilisation));
        sb.append(String.format("Registered customers: %d", customers.getAll().size()));
        return sb.toString();
    }

    private String revenueReport() {
        StringBuilder sb = new StringBuilder("\n===== REVENUE REPORT =====\n");
        sb.append(String.format("Rentals  - active: %d, completed: %d, cancelled: %d%n",
                rentals.count(RentalStatus.ACTIVE), rentals.count(RentalStatus.COMPLETED),
                rentals.count(RentalStatus.CANCELLED)));
        sb.append("Revenue from completed rentals by category:\n");
        for (Map.Entry<VehicleType, Double> e : rentals.revenueByType().entrySet()) {
            sb.append(String.format("  %-12s : %s%n", e.getKey(),
RentalService.money(e.getValue())));
        }
        sb.append(String.format("Total earned        : %s%n",
RentalService.money(rentals.completedRevenue())));
        sb.append(String.format("  of which late fees: %s%n",
RentalService.money(rentals.totalLateFees())));
        sb.append(String.format("Pending (active)    : %s",
RentalService.money(rentals.pendingRevenue())));
        return sb.toString();
    }
}
```

# 5. Output

```
Vehicle Rental - Terminal

============================================================
        RTPS VEHICLE RENTAL MANAGEMENT SYSTEM
============================================================

--- MAIN MENU ---
1. Vehicle Management
2. Customer Management
3. Rental Operations
4. Reports
0. Exit
Enter choice: 1

--- VEHICLE MANAGEMENT ---
1. View all vehicles
2. View available vehicles
3. Search vehicles
4. Add a vehicle
5. Update daily rate
6. Send to / bring back from maintenance
7. Remove a vehicle
0. Back
Enter choice: 1

ID        Type        Vehicle                      Plate         Rate/Day Status       Specifications
-------------------------------------------------------------------------------------------------------------------
CAR-101   Car         2023 Maruti Suzuki Swift VXi   TN30AB1101     1800.00 Available    5 seats, Manual, Petrol
CAR-102   Car         2024 Hyundai Creta SX          TN30CD2202     3200.00 Available    5 seats, Automatic, Diesel
CAR-103   Car         2022 Toyota Innova Crysta      TN30EF3303     4200.00 Available    7 seats, Manual, Diesel
CAR-104   Car         2024 Tata Nexon EV             TN30GH4404     2900.00 Maintenance  5 seats, Automatic, Electric
BIKE-101  Motorcycle  2023 Royal Enfield Classic 350 TN30JK5501     1200.00 Available    349 cc, helmet included
BIKE-102  Motorcycle  2024 TVS Apache RTR 160        TN30LM6602      700.00 Available    160 cc, helmet included
TRK-101   Truck       2023 Mahindra Bolero Pickup    TN30NP7701     3500.00 Available    1.5 ton capacity
TRK-102   Truck       2022 Tata Ace Gold             TN30QR8802     2200.00 Available    0.75 ton capacity
8 vehicle(s) listed.

--- VEHICLE MANAGEMENT ---
1. View all vehicles
2. View available vehicles
3. Search vehicles
4. Add a vehicle
5. Update daily rate
6. Send to / bring back from maintenance
7. Remove a vehicle
0. Back
Enter choice: 0

--- MAIN MENU ---
1. Vehicle Management
2. Customer Management
3. Rental Operations
4. Reports
0. Exit
Enter choice: 0

All data saved. Thank you for using the system. Goodbye!
```

*Figure 1: Main menu and list of all vehicles*

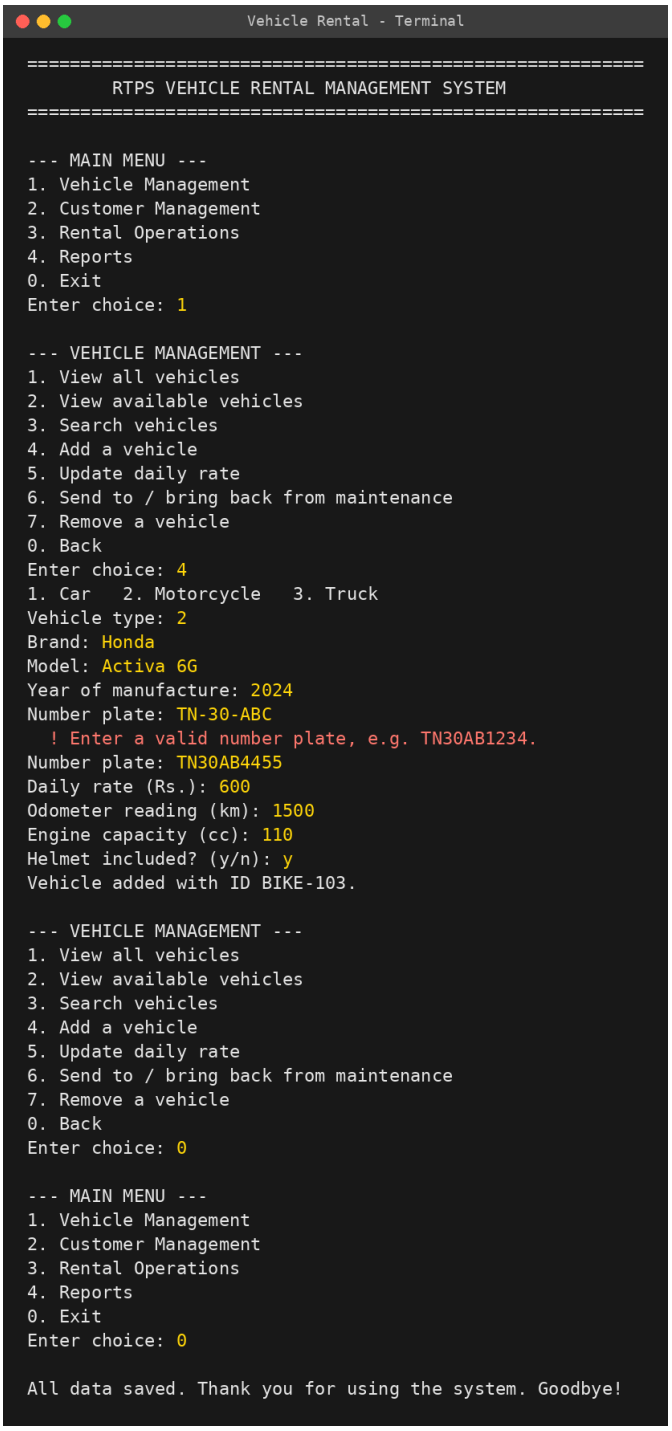



*Figure 2: Adding a new vehicle (number plate is validated)*

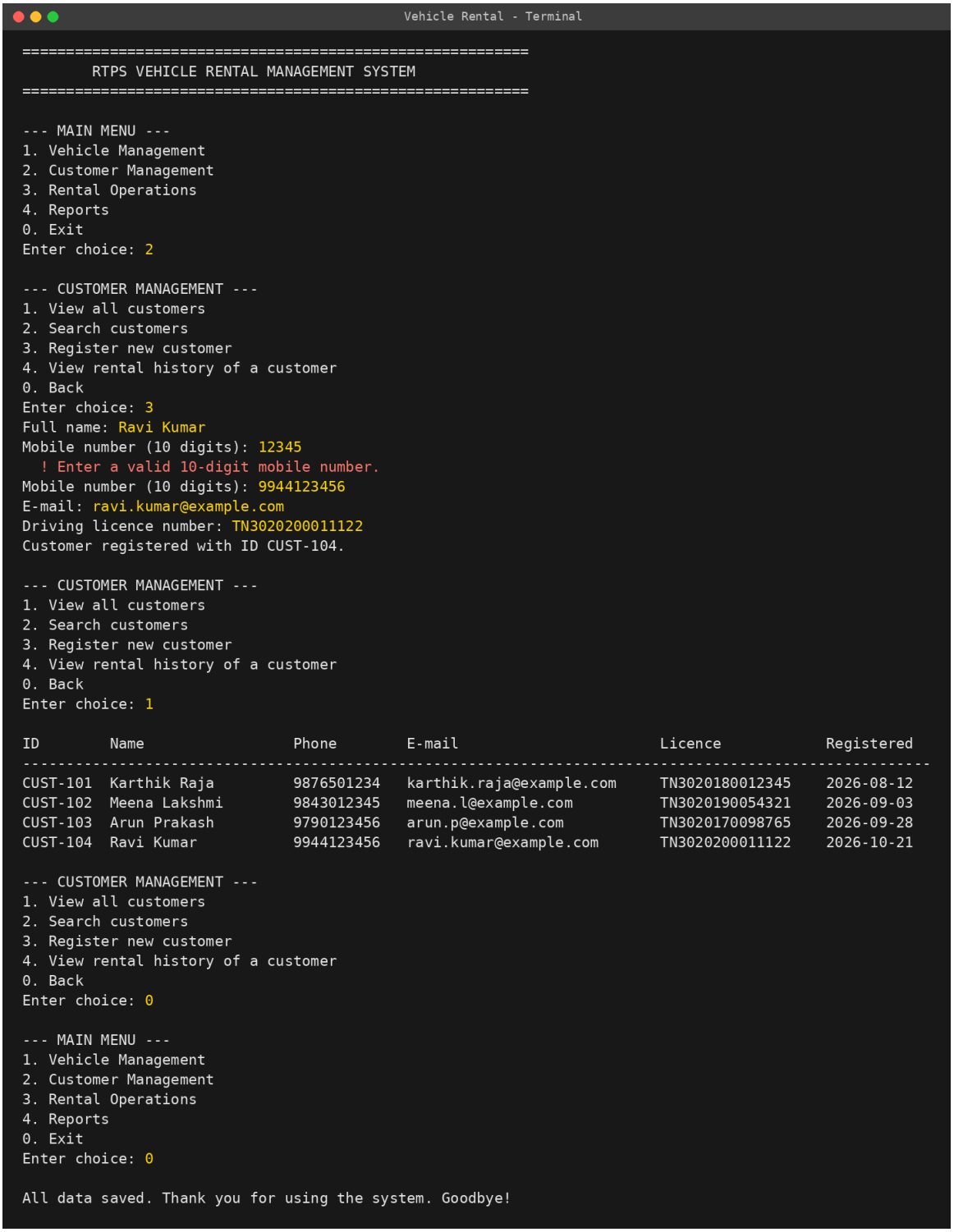



*Figure 3: Registering a customer (mobile number is validated)*

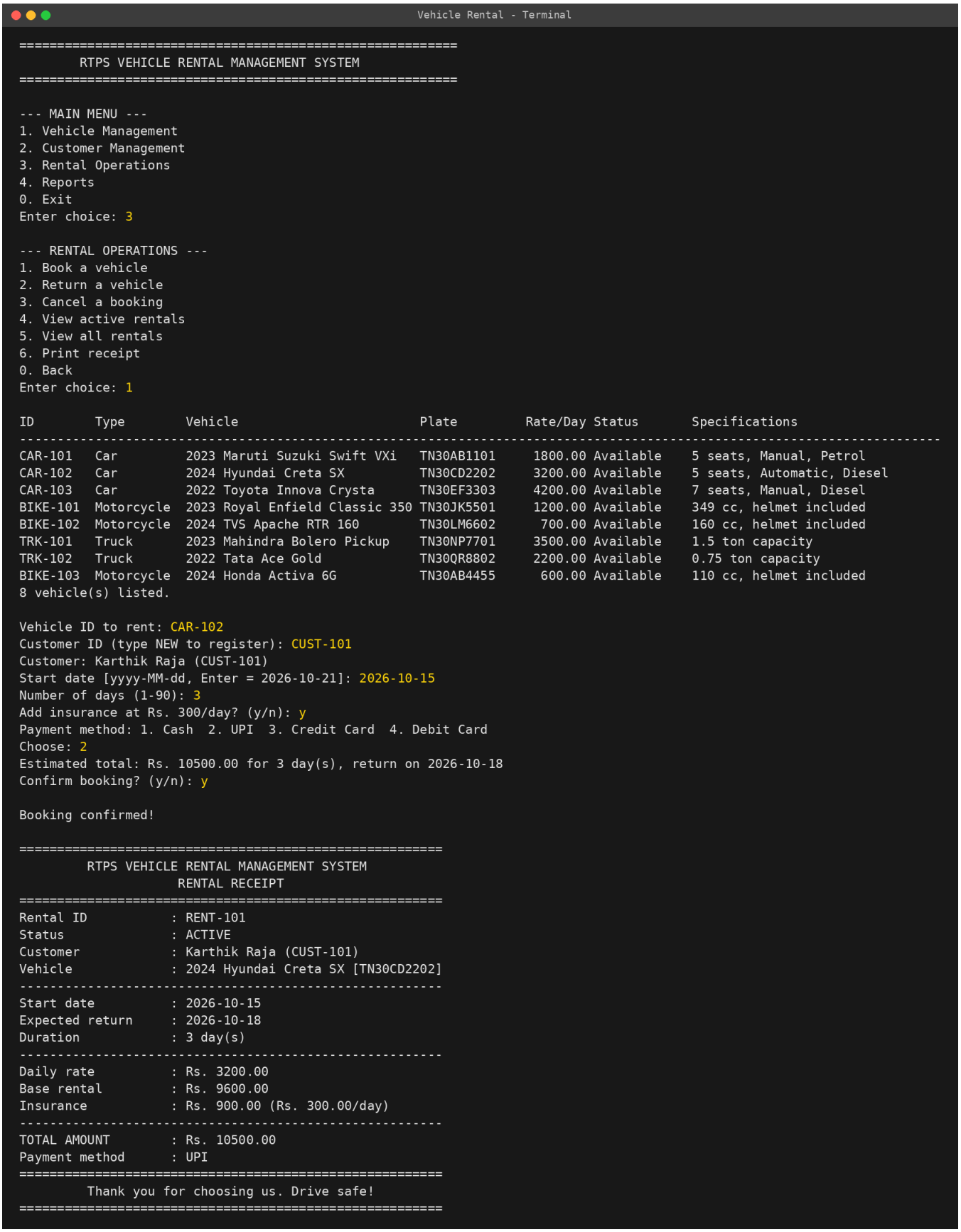



*Figure 4: Booking a car with insurance and viewing active rentals*

```
--- RENTAL OPERATIONS ---
1. Book a vehicle
2. Return a vehicle
3. Cancel a booking
4. View active rentals
5. View all rentals
6. Print receipt
0. Back
Enter choice: 4

Rental     Customer         Vehicle   Start       Due         Returned    Total (Rs.) Status
-----------------------------------------------------------------------------------------------
RENT-101   Karthik Raja     CAR-102   2026-10-15  2026-10-18  -              10500.00 Active

--- RENTAL OPERATIONS ---
1. Book a vehicle
2. Return a vehicle
3. Cancel a booking
4. View active rentals
5. View all rentals
6. Print receipt
0. Back
Enter choice: 0

--- MAIN MENU ---
1. Vehicle Management
2. Customer Management
3. Rental Operations
4. Reports
0. Exit
Enter choice: 0

All data saved. Thank you for using the system. Goodbye!
```

*Figure 4: Booking a car with insurance and viewing active rentals (continued, part 2 of 2)*

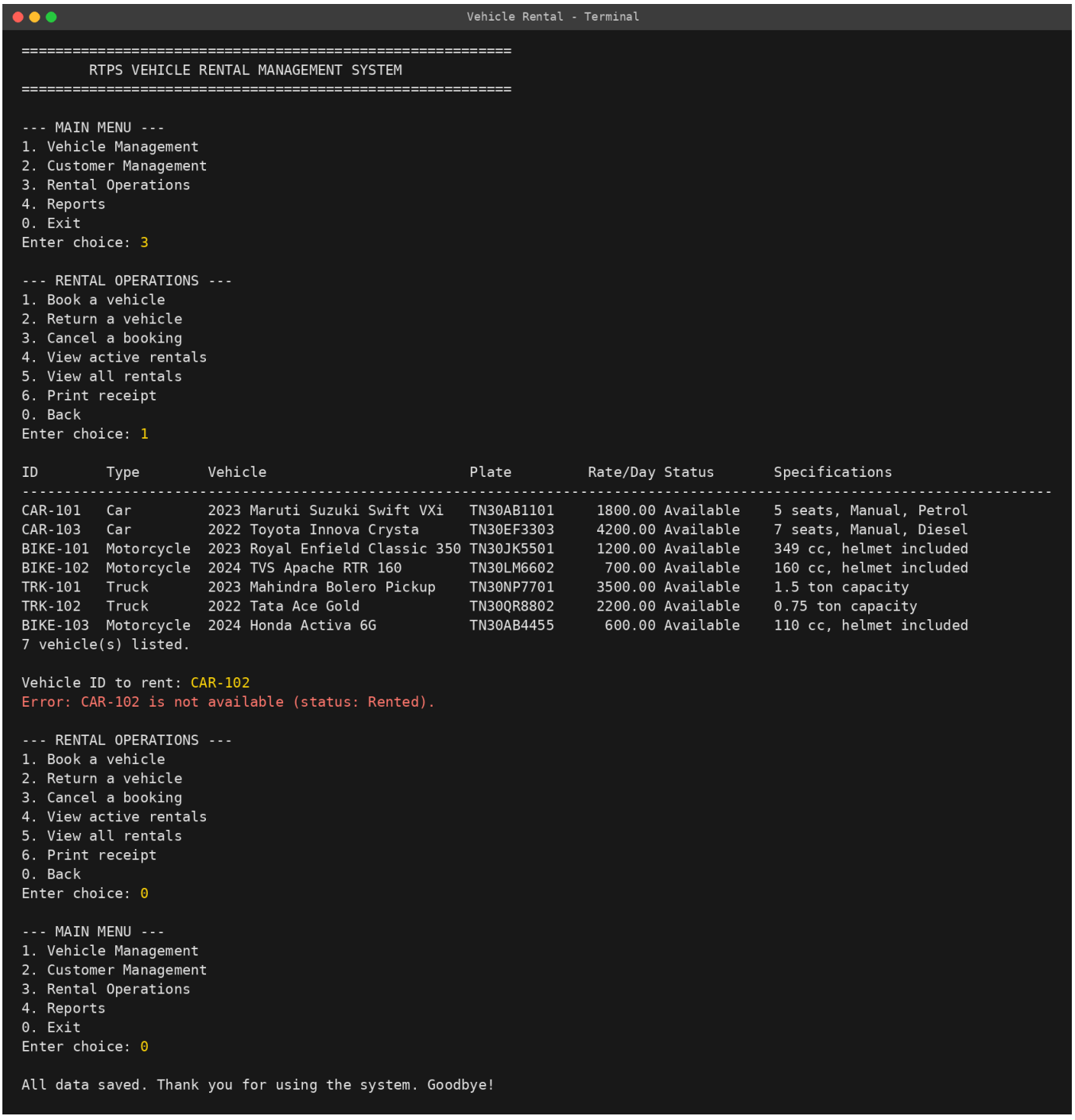



*Figure 5: Booking attempt on a vehicle that is already rented*

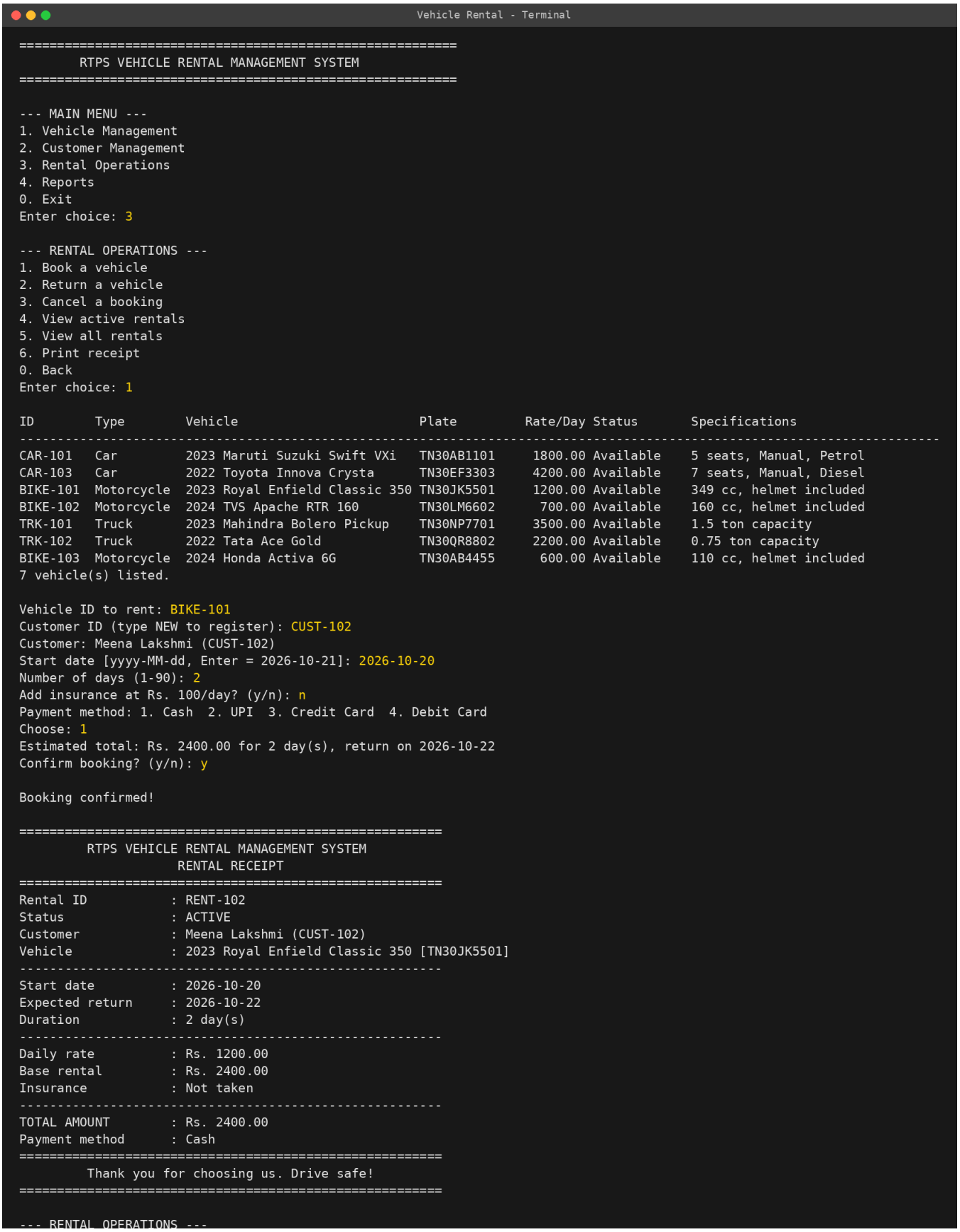



*Figure 6: Booking a motorcycle and cancelling the booking*

```
1. Book a vehicle
2. Return a vehicle
3. Cancel a booking
4. View active rentals
5. View all rentals
6. Print receipt
0. Back
Enter choice: 3
Rental ID to cancel: RENT-102
Reason: Customer changed plans
Booking cancelled; vehicle is available again.

--- RENTAL OPERATIONS ---
1. Book a vehicle
2. Return a vehicle
3. Cancel a booking
4. View active rentals
5. View all rentals
6. Print receipt
0. Back
Enter choice: 5

Rental     Customer         Vehicle   Start       Due         Returned    Total (Rs.) Status
-----------------------------------------------------------------------------------------------
RENT-101   Karthik Raja     CAR-102   2026-10-15  2026-10-18  -              10500.00 Active
RENT-102   Meena Lakshmi    BIKE-101  2026-10-20  2026-10-22  -               2400.00 Cancelled

--- RENTAL OPERATIONS ---
1. Book a vehicle
2. Return a vehicle
3. Cancel a booking
4. View active rentals
5. View all rentals
6. Print receipt
0. Back
Enter choice: 0

--- MAIN MENU ---
1. Vehicle Management
2. Customer Management
3. Rental Operations
4. Reports
0. Exit
Enter choice: 0

All data saved. Thank you for using the system. Goodbye!
```

*Figure 6: Booking a motorcycle and cancelling the booking (continued, part 2 of 2)*

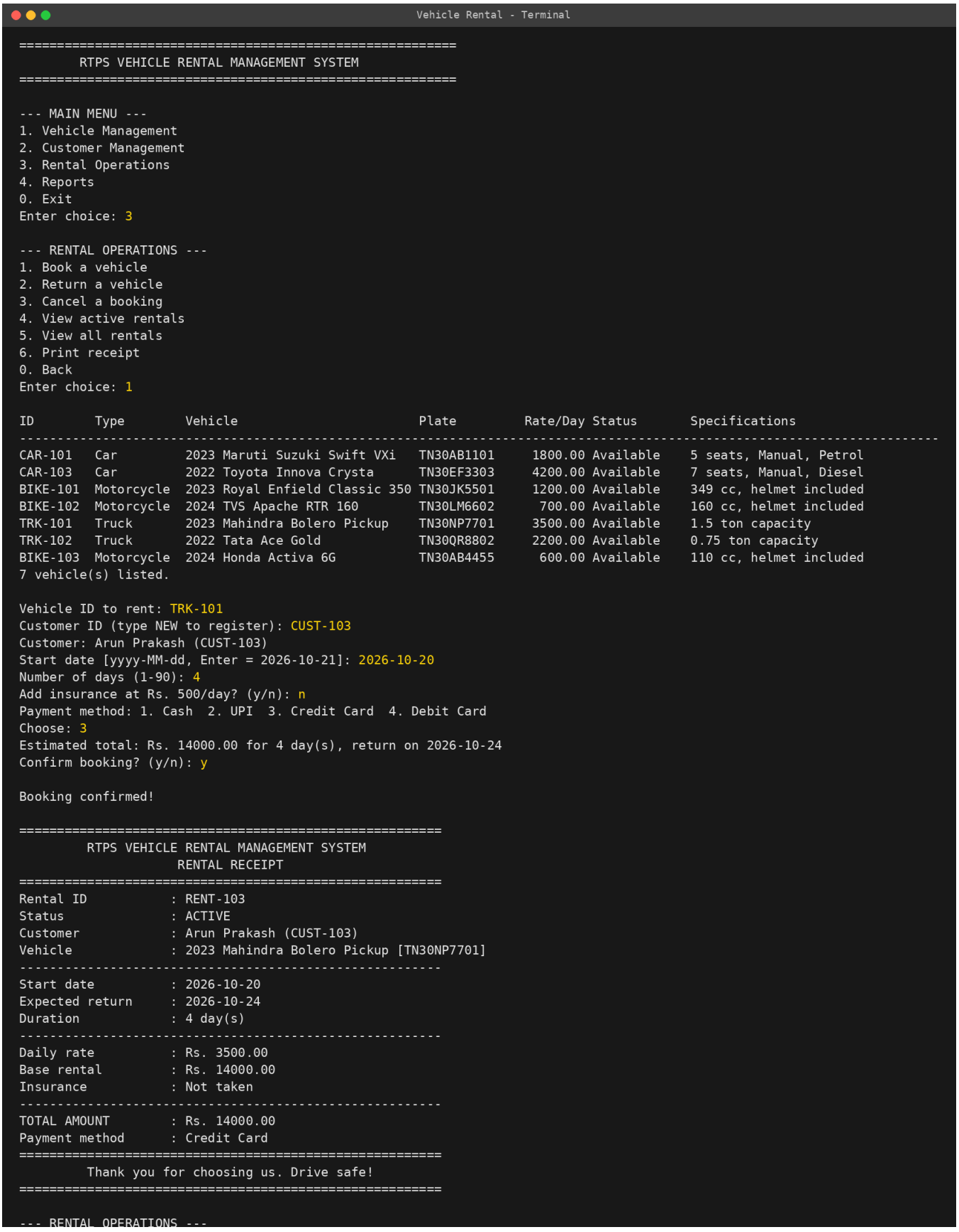



*Figure 7: Booking a truck*

```
1. Book a vehicle
2. Return a vehicle
3. Cancel a booking
4. View active rentals
5. View all rentals
6. Print receipt
0. Back
Enter choice: 0

--- MAIN MENU ---
1. Vehicle Management
2. Customer Management
3. Rental Operations
4. Reports
0. Exit
Enter choice: 0

All data saved. Thank you for using the system. Goodbye!
```

*Figure 7: Booking a truck (continued, part 2 of 2)*

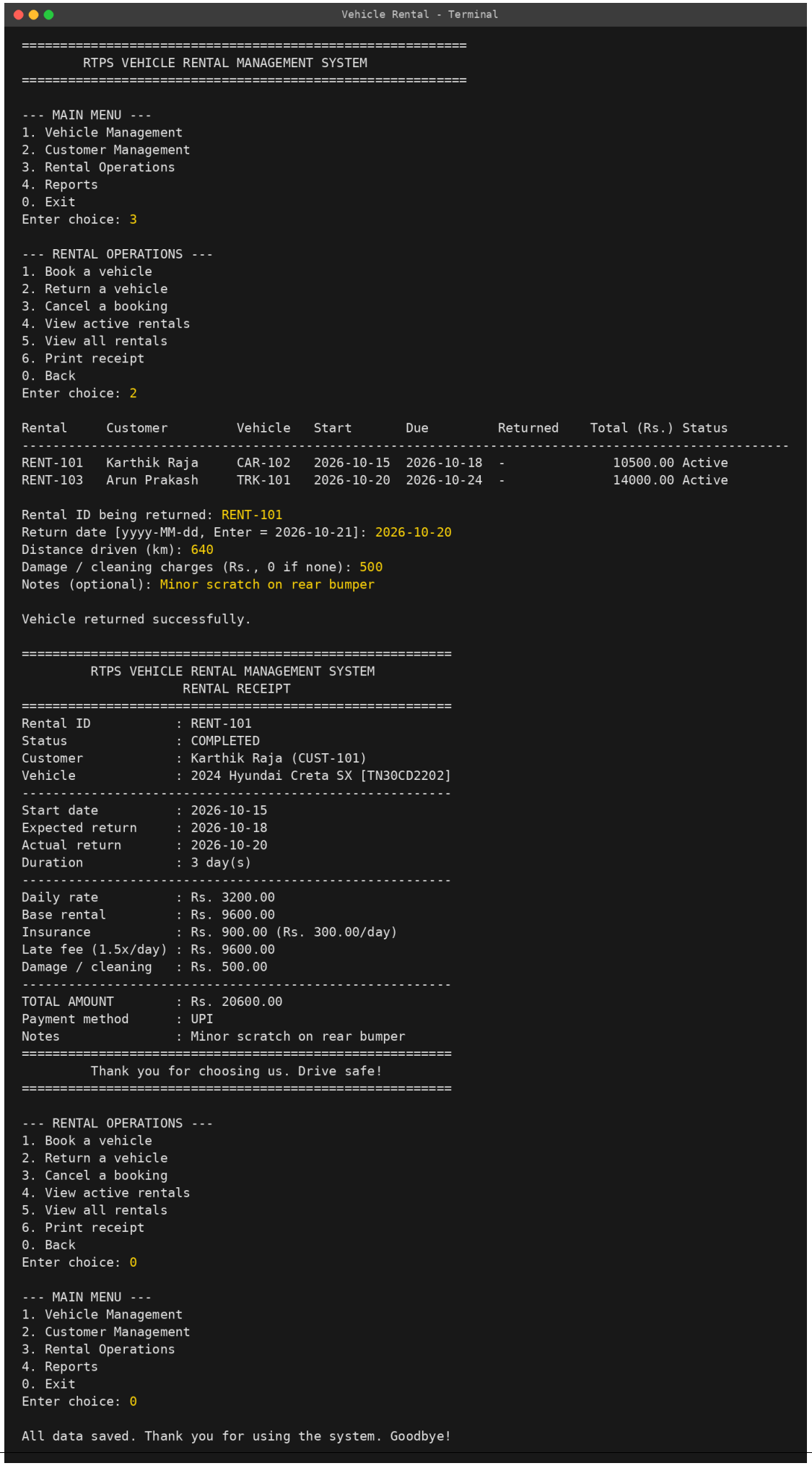

Vehicle Rental - Terminal
==========
RTPS VEHICLE RENTAL MANAGEMENT SYSTEM
==========
--- MAIN MENU ---
1. Vehicle Management
2. Customer Management
3. Rental Operations
4. Reports
0. Exit
Enter choice: 3
--- RENTAL OPERATIONS ---
1. Book a vehicle
2. Return a vehicle
3. Cancel a booking
4. View active rentals
5. View all rentals
6. Print receipt
0. Back
Enter choice: 2
Rental Customer Vehicle Start Due Returned Total (Rs.) Status
RENT-101 Karthik Raja CAR-102 2026-10-15 2026-10-18 - 10500.00 Active
RENT-103 Arun Prakash TRK-101 2026-10-20 2026-10-24 - 14000.00 Active
Rental ID being returned: RENT-101
Return date [yyyy-MM-dd, Enter = 2026-10-21]: 2026-10-20
Distance driven (km): 640
Damage / cleaning charges (Rs., 0 if none): 500
Notes (optional): Minor scratch on rear bumper
Vehicle returned successfully.
==========
RTPS VEHICLE RENTAL MANAGEMENT SYSTEM
RENTAL RECEIPT
==========
Rental ID : RENT-101
Status : COMPLETED
Customer : Karthik Raja (CUST-101)
Vehicle : 2024 Hyundai Creta SX [TN30CD2202]
----------
Start date : 2026-10-15
Expected return : 2026-10-18
Actual return : 2026-10-20
Duration : 3 day(s)
----------
Daily rate : Rs. 3200.00
Base rental : Rs. 9600.00
Insurance : Rs. 900.00 (Rs. 300.00/day)
Late fee (1.5x/day) : Rs. 9600.00
Damage / cleaning : Rs. 500.00
----------
TOTAL AMOUNT : Rs. 20600.00
Payment method : UPI
Notes : Minor scratch on rear bumper
==========
Thank you for choosing us. Drive safe!
==========
--- RENTAL OPERATIONS ---
1. Book a vehicle
2. Return a vehicle
3. Cancel a booking
4. View active rentals
5. View all rentals
6. Print receipt
0. Back
Enter choice: 0
--- MAIN MENU ---
1. Vehicle Management
2. Customer Management
3. Rental Operations
4. Reports
0. Exit
Enter choice: 0
All data saved. Thank you for using the system. Goodbye!

*Figure 8: Returning a vehicle two days late with damage charge*

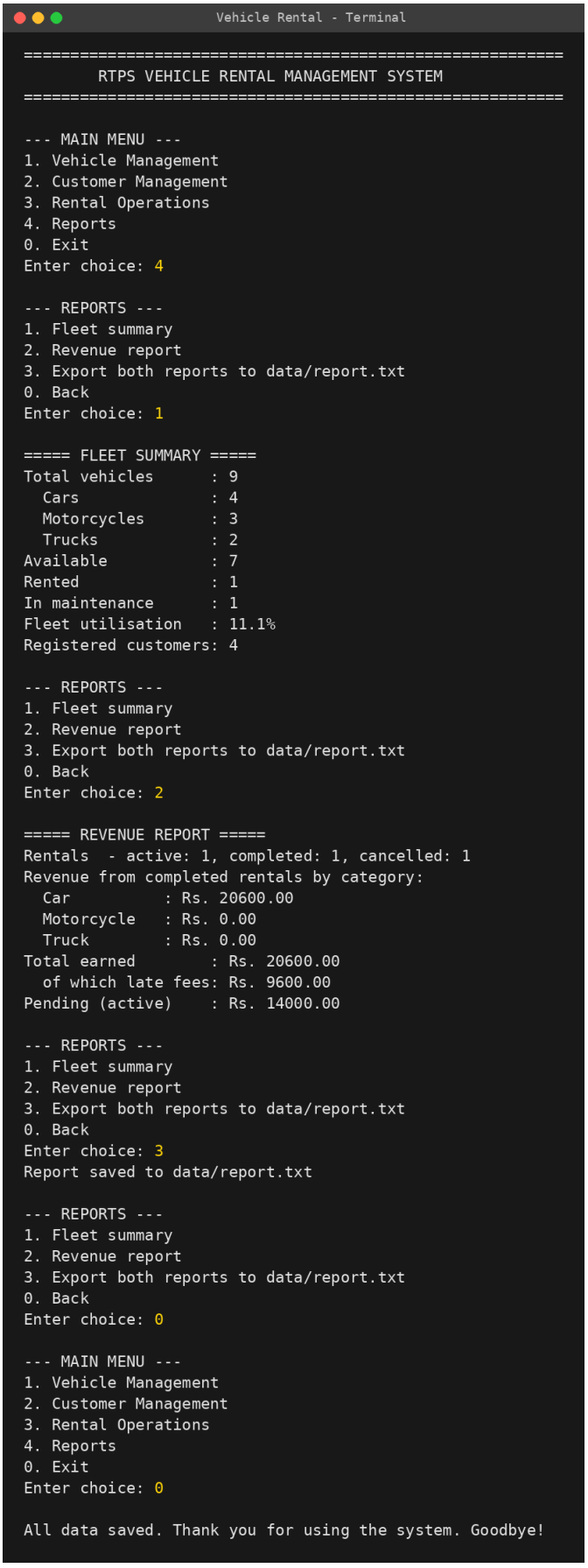



*Figure 9: Fleet summary, revenue report and export*

# 6. Conclusion

The Vehicle Rental Management System provides an efficient and organised way to manage a rental fleet, its customers and its bookings from the terminal. The application simplifies the process of booking, returning and cancelling rentals while keeping accurate records of every transaction.

By implementing Java concepts such as abstract classes, interfaces, inheritance, polymorphism, encapsulation, custom exception handling, collections and file handling, the project demonstrates how these concepts can be combined to build a complete and maintainable application. The layered structure keeps the user interface, business rules and data storage separate, which makes the program easy to extend.

Overall, the project meets its objective of reducing manual record-keeping and calculation errors, and it can be extended in future with features such as a graphical interface, a database, online payments and rental reminders.